

Python Basics - Tài liệu tham khảo cơ bản

1. BIẾN & KIỂU DỮ LIỆU

Khai báo biến

```
python

x = 5          # Integer
y = 3.14       # Float
name = "Python" # String
is_valid = True # Boolean
nothing = None  # None (null)

# Gán nhiều biến
a, b, c = 1, 2, 3
x = y = z = 0
```

Kiểu dữ liệu cơ bản

```
python

type(x)          # Kiểm tra kiểu dữ liệu

# Chuyển đổi kiểu
int("123")        # → 123
float("3.14")     # → 3.14
str(123)          # → "123"
bool(0)           # → False
bool(1)           # → True
```

Số (Numbers)

```
python
```

Integer

x = 10

y = -5

z = 0

Float

pi = 3.14159

e = 2.718

Complex

c = 3 + 4j

Toán tử

a + b # Cộng

a - b # Trừ

a * b # Nhân

a / b # Chia (float)

a // b # Chia lấy nguyên

a % b # Chia lấy dư

a ** b # Lũy thừa

Hàm toán học

abs(-5) # Giá trị tuyệt đối: 5

round(3.7) # Làm tròn: 4

pow(2, 3) # 2^3 = 8

max(1, 5, 3) # 5

min(1, 5, 3) # 1

2. STRING (CHUỖI)

Tạo string

python

```
s1 = 'Hello'
s2 = "World"
s3 = """Multi
line
string"""
s4 = """Triple quotes"""

# Raw string (không escape)
path = r"C:\Users\name"
```

Truy cập

```
python

s = "Python"
s[0]          # 'P' (ký tự đầu)
s[-1]         # 'n' (ký tự cuối)
s[1:4]        # 'yth' (từ vị trí 1-3)
s[:3]         # 'Pyt' (3 ký tự đầu)
s[3:]         # 'hon' (từ vị trí 3 đến cuối)
s[::-2]       # 'Pto' (mỗi ký tự thứ 2)
s[::-1]       # 'nohtyP' (đảo ngược)
```

Phương thức String

```
python
```

```
s = " Hello World "
```

```
# Case
```

```
s.upper()      # " HELLO WORLD "
```

```
s.lower()      # " hello world "
```

```
s.capitalize() # " hello world "
```

```
s.title()      # " Hello World "
```

```
s.swapcase()   # " hELLO wORLD "
```

```
# Trim
```

```
s.strip()      # "Hello World"
```

```
s.lstrip()     # "Hello World "
```

```
s.rstrip()     # " Hello World"
```

```
# Tìm kiếm
```

```
s.find("World") # 8 (vị trí xuất hiện)
```

```
s.find("xyz")   # -1 (không tìm thấy)
```

```
s.index("World") # 8 (giống find nhưng raise lỗi nếu không tìm thấy)
```

```
s.count("l")    # 3 (đếm số lần xuất hiện)
```

```
"World" in s    # True
```

```
"xyz" not in s  # True
```

```
# Kiểm tra
```

```
s.startswith("Hello") # True
```

```
s.endswith("World")   # True
```

```
s.isdigit()          # False
```

```
s.isalpha()          # False
```

```
s.isalnum()          # False
```

```
s.isspace()          # False
```

```
s.islower()          # False
```

```
s.isupper()          # False
```

```
# Thay thế
```

```
s.replace("World", "Python") # " Hello Python "
```

```
# Tách & nối
```

```
s.split()          # ["Hello", "World"]
```

```
s.split(",")       # Tách theo dấu phẩy
```

```
"-".join(["a", "b", "c"]) # "a-b-c"
```

```
# Format
```

```
name = "John"
```

```
age = 25
```

```
f"My name is {name}, {age} years old" #f-string (Python 3.6+)
```

```
"My name is {}, {} years old".format(name, age)
```

```
"My name is %, %d years old" % (name, age)
```

Khác

```
len(s) # Độ dài
```

```
s.center(20, "-") # Căn giữa với padding
```

```
s.ljust(20, "-") # Căn trái
```

```
s.rjust(20, "-") # Căn phải
```

```
s.zfill(10) # Thêm số 0 vào đầu
```

3. LIST (DANH SÁCH)

Tạo list

python

```
lst = [1, 2, 3, 4, 5]
```

```
mixed = [1, "hello", 3.14, True]
```

```
empty = []
```

```
nested = [[1, 2], [3, 4], [5, 6]]
```

Truy cập

python

```
lst = [10, 20, 30, 40, 50]
```

```
lst[0] # 10
```

```
lst[-1] # 50
```

```
lst[1:3] # [20, 30]
```

```
lst[:3] # [10, 20, 30]
```

```
lst[2:] # [30, 40, 50]
```

```
lst[::2] # [10, 30, 50]
```

```
lst[::-1] # [50, 40, 30, 20, 10]
```

Thêm & xóa

python

```
lst = [1, 2, 3]
```

Thêm

```
lst.append(4)      # [1, 2, 3, 4]
```

```
lst.insert(1, 99)  # [1, 99, 2, 3, 4]
```

```
lst.extend([5, 6]) # [1, 99, 2, 3, 4, 5, 6]
```

```
lst += [7, 8]      # [1, 99, 2, 3, 4, 5, 6, 7, 8]
```

Xóa

```
lst.remove(99)      # Xóa giá trị 99 (lần đầu xuất hiện)
```

```
lst.pop()           # Xóa & trả về phần tử cuối
```

```
lst.pop(0)          # Xóa & trả về phần tử vị trí 0
```

```
del lst[1]          # Xóa phần tử vị trí 1
```

```
del lst[1:3]        # Xóa slice
```

```
lst.clear()         # Xóa tất cả
```

Phương thức khác

python

```
lst = [3, 1, 4, 1, 5, 9, 2]
```

Sắp xếp

```
lst.sort()          # [1, 1, 2, 3, 4, 5, 9] (sửa tại chỗ)
```

```
lst.sort(reverse=True) # [9, 5, 4, 3, 2, 1, 1] (giảm dần)
```

```
sorted(lst)         # Trả về list mới đã sort
```

```
lst.reverse()       # Đảo ngược tại chỗ
```

```
lst[::-1]           # Trả về list đảo ngược (không sửa gốc)
```

Tìm kiếm

```
lst.index(4)        # 2 (vị trí của 4)
```

```
lst.count(1)        # 2 (số lần xuất hiện)
```

```
4 in lst            # True
```

```
10 not in lst       # True
```

Khác

```
len(lst)            # Độ dài
```

```
min(lst)            # Giá trị nhỏ nhất
```

```
max(lst)            # Giá trị lớn nhất
```

```
sum(lst)            # Tổng
```

```
lst.copy()          # Copy list
```

List Comprehension

```
python

# Cú pháp: [expression for item in iterable if condition]

# Ví dụ
[x**2 for x in range(5)]           # [0, 1, 4, 9, 16]
[x for x in range(10) if x % 2 == 0] # [0, 2, 4, 6, 8]
[x*2 if x > 5 else x for x in range(10)]

# Nested
[[i*j for j in range(3)] for i in range(3)]
# [[0, 0, 0], [0, 1, 2], [0, 2, 4]]
```

4. TUPLE (BỘ)

Tạo tuple

```
python

t = (1, 2, 3)
t = 1, 2, 3           # Không cần ngoặc
single = (1,)         # Tuple 1 phần tử (có dấu phẩy)
empty = ()
mixed = (1, "hello", 3.14)
```

Đặc điểm

```
python
```

Tuple KHÔNG THỂ THAY ĐỔI (immutable)

```
t = (1, 2, 3)
```

```
t[0] = 10          # ❌ Lỗi TypeError
```

Truy cập giống list

```
t[0]              # 1
```

```
t[-1]            # 3
```

```
t[1:3]           # (2, 3)
```

Phương thức

```
t.count(2)        # Đếm số lần xuất hiện
```

```
t.index(3)        # Vị trí của 3
```

```
len(t)            # Độ dài
```

Unpacking

python

```
t = (1, 2, 3)
```

```
a, b, c = t        # a=1, b=2, c=3
```

```
a, *rest = t       # a=1, rest=[2, 3]
```

```
*first, last = t    # first=[1, 2], last=3
```

5. DICTIONARY (TỪ ĐIỂN)

Tạo dict

python

```
d = {"name": "John", "age": 25, "city": "Hanoi"}
```

```
d = dict(name="John", age=25)
```

```
empty = {}
```

Truy cập

python

```
d = {"name": "John", "age": 25}

d["name"]          # "John"
d.get("name")      # "John"
d.get("address", "N/A") # "N/A" (không tìm thấy key)
"name" in d        # True
"address" not in d  # True
```

Thêm & sửa

```
python

d = {"name": "John"}

d["age"] = 25      # Thêm key mới
d["name"] = "Jane" # Sửa giá trị
d.update({"city": "Hanoi", "age": 26}) # Cập nhật nhiều
```

Xóa

```
python

d = {"name": "John", "age": 25, "city": "Hanoi"}

del d["age"]      # Xóa key "age"
d.pop("city")     # Xóa & trả về giá trị "Hanoi"
d.popitem()       # Xóa & trả về cặp key-value cuối
d.clear()         # Xóa tất cả
```

Phương thức

```
python
```

```
d = {"name": "John", "age": 25, "city": "Hanoi"}

d.keys()          # dict_keys(['name', 'age', 'city'])
d.values()         # dict_values(['John', 25, 'Hanoi'])
d.items()          # dict_items([('name', 'John'), ...])

list(d.keys())     # ['name', 'age', 'city']
list(d.values())   # ['John', 25, 'Hanoi']

len(d)            # 3
d.copy()           # Copy dict

# Setdefault
d.setdefault("country", "Vietnam") # Thêm nếu chưa có
```

Dict Comprehension

```
python

{x: x**2 for x in range(5)}
# {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

{k: v.upper() for k, v in d.items() if isinstance(v, str)}
```

6. SET (TẬP HỢP)

Tạo set

```
python

s = {1, 2, 3, 4, 5}
s = set([1, 2, 2, 3]) # {1, 2, 3} (tự động loại trùng)
empty = set()        # Set rỗng (KHÔNG dùng {})
```

Đặc điểm

```
python

# Set KHÔNG có thứ tự, KHÔNG có phần tử trùng
s = {3, 1, 2}
print(s)          # {1, 2, 3} (tự động sắp xếp khi in)
```

Thêm & xóa

python

```
s = {1, 2, 3}
```

```
s.add(4)           # {1, 2, 3, 4}
```

```
s.update([5, 6])   # {1, 2, 3, 4, 5, 6}
```

```
s.remove(3)        # Xóa 3 (lỗi nếu không tồn tại)
```

```
s.discard(3)       # Xóa 3 (không lỗi nếu không tồn tại)
```

```
s.pop()            # Xóa & trả về phần tử bất kỳ
```

```
s.clear()          # Xóa tất cả
```

Phép toán tập hợp

python

```
a = {1, 2, 3, 4}
```

```
b = {3, 4, 5, 6}
```

Hợp (Union)

```
a | b              # {1, 2, 3, 4, 5, 6}
```

```
a.union(b)
```

Giao (Intersection)

```
a & b              # {3, 4}
```

```
a.intersection(b)
```

Hiệu (Difference)

```
a - b              # {1, 2}
```

```
a.difference(b)
```

Hiệu đối xứng (Symmetric Difference)

```
a ^ b              # {1, 2, 5, 6}
```

```
a.symmetric_difference(b)
```

Kiểm tra

```
a.issubset(b)      # a có là tập con của b?
```

```
a.issuperset(b)    # a có là tập cha của b?
```

```
a.isdisjoint(b)    # a và b có rời nhau không?
```

7. TOÁN TỬ

Toán tử số học

python

```
+    # Cộng
-    # Trừ
*    # Nhân
/    # Chia (float)
//   # Chia lấy nguyên
%    # Chia lấy dư
**   # Lũy thừa
```

Toán tử so sánh

python

```
==   # Bằng
!=   # Khác
>    # Lớn hơn
<    # Nhỏ hơn
>=   # Lớn hơn hoặc bằng
<=   # Nhỏ hơn hoặc bằng
```

Toán tử logic

python

```
and   # Và
or    # Hoặc
not   # Phủ định

# Ví dụ
x > 0 and x < 10
x == 5 or x == 10
not (x > 5)
```

Toán tử gán

python

```
=      # Gán
+=     # x += 1 → x = x + 1
-=     # x -= 1 → x = x - 1
*=     # x *= 2 → x = x * 2
/=     # x /= 2 → x = x / 2
//=    # x //= 2 → x = x // 2
%=     # x %= 2 → x = x % 2
**=    # x **= 2 → x = x ** 2
```

Toán tử membership

```
python

in      # x in list
not in  # x not in list
```

Toán tử identity

```
python

is      # x is y (cùng object)
is not  # x is not y
```

8. CẤU TRÚC ĐIỀU KHIỂN

If-Elif-Else

```
python

x = 10

if x > 10:
    print("Lớn hơn 10")
elif x == 10:
    print("Bằng 10")
else:
    print("Nhỏ hơn 10")

# One-liner (Ternary)
result = "Even" if x % 2 == 0 else "Odd"
```

Match-Case (Python 3.10+)

```
python

status = 200

match status:
    case 200:
        print("OK")
    case 404:
        print("Not Found")
    case 500:
        print("Server Error")
    case _:
        print("Unknown")
```

9. VÒNG LẶP

For Loop

```
python
```

```
# Lặp qua list
for item in [1, 2, 3]:
    print(item)

# Lặp qua range
for i in range(5):      # 0, 1, 2, 3, 4
    print(i)

for i in range(1, 6):   # 1, 2, 3, 4, 5
    print(i)

for i in range(0, 10, 2): # 0, 2, 4, 6, 8
    print(i)

# Lặp qua string
for char in "Python":
    print(char)

# Lặp qua dict
d = {"a": 1, "b": 2}
for key in d:
    print(key, d[key])

for key, value in d.items():
    print(key, value)

# Enumerate (lấy cả index)
for i, value in enumerate(['a', 'b', 'c']):
    print(i, value)      # 0 a, 1 b, 2 c

# Zip (lặp song song)
names = ['John', 'Jane']
ages = [25, 30]
for name, age in zip(names, ages):
    print(name, age)
```

While Loop

```
python
```

```

i = 0
while i < 5:
    print(i)
    i += 1

# While với else
i = 0
while i < 3:
    print(i)
    i += 1
else:
    print("Done")      # Chạy khi while kết thúc bình thường

```

Break & Continue

```

python

# Break - Thoát vòng lặp
for i in range(10):
    if i == 5:
        break
    print(i)      # 0, 1, 2, 3, 4

# Continue - Bỏ qua iteration hiện tại
for i in range(5):
    if i == 2:
        continue
    print(i)      # 0, 1, 3, 4

# Pass - Không làm gì
for i in range(5):
    pass          # Placeholder

```

10. HÀM (FUNCTIONS)

Định nghĩa hàm

```
python
```

```
def greet():
    print("Hello!")

def greet_name(name):
    print(f"Hello, {name}!")

def add(a, b):
    return a + b

# Gọi hàm
greet()
greet_name("John")
result = add(3, 5)
```

Tham số mặc định

```
python

def greet(name="Guest"):
    print(f"Hello, {name}!")

greet()           # Hello, Guest!
greet("John")     # Hello, John!
```

Tham số từ khóa

```
python

def introduce(name, age, city):
    print(f"{name}, {age} years old, from {city}")

introduce(name="John", age=25, city="Hanoi")
introduce(age=25, city="Hanoi", name="John") # Thứ tự tùy ý
```

*args & **kwargs

```
python
```

*# *args - Số lượng tham số tùy ý (tuple)*

```
def sum_all(*args):  
    return sum(args)
```

```
sum_all(1, 2, 3)      # 6
```

```
sum_all(1, 2, 3, 4, 5)  # 15
```

*# **kwargs - Tham số từ khóa tùy ý (dict)*

```
def print_info(**kwargs):  
    for key, value in kwargs.items():  
        print(f'{key}: {value}')
```

```
print_info(name="John", age=25, city="Hanoi")
```

Lambda (Anonymous Function)

python

Cú pháp: lambda arguments: expression

```
square = lambda x: x ** 2
```

```
square(5)      # 25
```

```
add = lambda x, y: x + y
```

```
add(3, 5)      # 8
```

Dùng với map, filter, sorted

```
numbers = [1, 2, 3, 4, 5]
```

```
squared = list(map(lambda x: x**2, numbers))
```

```
evens = list(filter(lambda x: x % 2 == 0, numbers))
```

Docstring

python

```
def add(a, b):  
    """  
    Cộng hai số.  
  
    Args:  
        a: Số thứ nhất  
        b: Số thứ hai  
  
    Returns:  
        Tổng của a và b  
    """  
    return a + b  
  
help(add)           # Xem docstring
```

11. XỬ LÝ NGOẠI LỆ

Try-Except

```
python
```

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Không thể chia cho 0!")

# Nhiều except
try:
    # code
except ValueError:
    print("Lỗi giá trị")
except TypeError:
    print("Lỗi kiểu")
except Exception as e:
    print(f"Lỗi khác: {e}")

# Else & Finally
try:
    result = 10 / 2
except ZeroDivisionError:
    print("Lỗi chia 0")
else:
    print("Thành công!") # Chạy nếu không có lỗi
finally:
    print("Luôn chạy") # Luôn chạy
```

Raise Exception

```
python

def check_age(age):
    if age < 0:
        raise ValueError("Tuổi không thể âm!")
    return age

try:
    check_age(-5)
except ValueError as e:
    print(e)
```

12. FILE I/O

Đọc file

```
python

# Cách 1: Với with (tự động đóng file)
with open("file.txt", "r", encoding="utf-8") as f:
    content = f.read()    # Đọc toàn bộ

with open("file.txt", "r") as f:
    lines = f.readlines() # Đọc thành list các dòng

with open("file.txt", "r") as f:
    for line in f:        # Đọc từng dòng
        print(line.strip())

# Cách 2: Không dùng with (phải đóng thủ công)
f = open("file.txt", "r")
content = f.read()
f.close()
```

Ghi file

```
python

# Ghi đè (overwrite)
with open("file.txt", "w", encoding="utf-8") as f:
    f.write("Hello World\n")
    f.write("Python Programming")

# Ghi thêm (append)
with open("file.txt", "a") as f:
    f.write("\nNew line")

# Ghi nhiều dòng
lines = ["Line 1\n", "Line 2\n", "Line 3\n"]
with open("file.txt", "w") as f:
    f.writelines(lines)
```

Mode

```
python
```

```
"r"    # Read (mặc định)
"w"    # Write (ghi đè)
"a"    # Append (ghi thêm)
"x"    # Create (tạo mới, lỗi nếu đã tồn tại)
"r+"   # Read + Write
"w+"   # Write + Read
"rb"   # Read binary
"wb"   # Write binary
```

13. MODULES & IMPORT

Import

```
python

import math
print(math.pi)          # 3.141592...
print(math.sqrt(16))    # 4.0

# Import với alias
import numpy as np
arr = np.array([1, 2, 3])

# Import specific
from math import pi, sqrt
print(pi)
print(sqrt(16))

# Import tất cả (không khuyến khích)
from math import *
```

Modules phổ biến

```
python
```

```
import math      # Toán học
import random    # Ngẫu nhiên
import datetime  # Ngày giờ
import os        # Hệ điều hành
import sys       # Hệ thống Python
import json      # JSON
import re        # Regular Expression
```

14. BUILT-IN FUNCTIONS

Input/Output

```
python

print("Hello")      # In ra màn hình
name = input("Your name: ") # Nhập từ bàn phím (string)
age = int(input("Age: "))  # Nhập số
```

Type Conversion

```
python

int()      # Chuyển sang integer
float()    # Chuyển sang float
str()      # Chuyển sang string
bool()     # Chuyển sang boolean
list()     # Chuyển sang list
tuple()    # Chuyển sang tuple
set()      # Chuyển sang set
dict()     # Chuyển sang dict
```

Math & Numbers

```
python
```

```
abs(-5)      # Giá trị tuyệt đối: 5
round(3.7)   # Làm tròn: 4
pow(2, 3)    # Lũy thừa: 8
min(1, 5, 3) # Nhỏ nhất: 1
max(1, 5, 3) # Lớn nhất: 5
sum([1,2,3]) # Tổng: 6
divmod(10, 3) # (3, 1) - thương và dư
```

Sequences

```
python

len([1,2,3])      # Độ dài: 3
range(5)           # 0, 1, 2, 3, 4
sorted([3,1,2])    # [1, 2, 3]
reversed([1,2,3])  # iterator đảo ngược
enumerate(['a','b']) # iterator với index
zip([1,2], ['a','b']) # iterator ghép cặp
all([True, True])  # True nếu tất cả True
any([False, True]) # True nếu có ít nhất 1 True
```

Functional

```
python

map(func, iterable)    # Áp dụng func cho mỗi phần tử
filter(func, iterable) # Lọc phần tử
reduce(func, iterable)  # Tích lũy (cần import từ functools)

# Ví dụ
list(map(lambda x: x**2, [1,2,3])) # [1, 4, 9]
list(filter(lambda x: x>2, [1,2,3,4])) # [3, 4]
```

Object & Variables

```
python
```

```
type(obj)      # Kiểu dữ liệu
id(obj)        # ID của object
isinstance(obj, type) # Kiểm tra kiểu
dir(obj)       # Danh sách thuộc tính/phương thức
help(obj)      # Hướng dẫn
vars(obj)      # __dict__ của object
```

15. TIPS & TRICKS

Swap variables

```
python

a, b = b, a
```

Multiple assignment

```
python

x = y = z = 0
a, b, c = 1, 2, 3
```

Ternary operator

```
python

result = "Even" if x % 2 == 0 else "Odd"
```

Check empty

```
python

if not my_list:      # Kiểm tra list rỗng
    print("Empty")

if my_list:          # Kiểm tra list không rỗng
    print("Not empty")
```

Merge dictionaries

```
python
```

```
d1 = {"a": 1, "b": 2}
d2 = {"c": 3, "d": 4}
merged = {**d1, **d2}      # Python 3.5+
merged = d1 | d2           # Python 3.9+
```

Flatten list

```
python

nested = [[1, 2], [3, 4], [5, 6]]
flat = [item for sublist in nested for item in sublist]
# [1, 2, 3, 4, 5, 6]
```

Remove duplicates

```
python

lst = [1, 2, 2, 3, 3, 4]
unique = list(set(lst))    # Mất thứ tự
unique = list(dict.fromkeys(lst))
```